Parallel & Distributed Computing

NETWORKS OF WORKSTATIONS (DISTRIBUTED MEMORY)

Dr. Muzamil Ahmed (Assistant Professor)
Namal University Mianwali

Lesson from Quran

Lecture Outline

- Workstation
- Network of Workstations (NOW)
- Architecture & Communication Mechanism in NOW
- Berkeley NOW Project (Case Study)
- NOW vs Multiprocessors
- Distributed Memory
- Data Distribution and Partitioning
- Issues in Distributed Memory
- Shared vs Distributed Memory

Workstation

- Workstation, a high-performance computer system as compared to mainstream personal computers, that is basically designed for a single user and has ***advanced graphics capabilities***, ***large storage capacity***, and a ***powerful central processing unit***.
- A workstation is ***more capable than a personal computer*** (PC) but is ***less advanced than a server*** (which can manage a large network of peripheral PCs or workstations and handle immense data-processing and reporting tasks).

Definition - Networks of Workstations

- A **Network of Workstations (NOW)** is:
  - A collection of interconnected, autonomous computers (workstations) that work together to perform parallel or distributed computing tasks over a high-speed local area network (LAN).
- **Key Aspects:**
  - Each node is a **fully functional workstation** (CPU, memory, OS).
  - All nodes are connected via **fast communication links**.
  - Nodes collaborate as if they were part of a **single, unified computing system**.

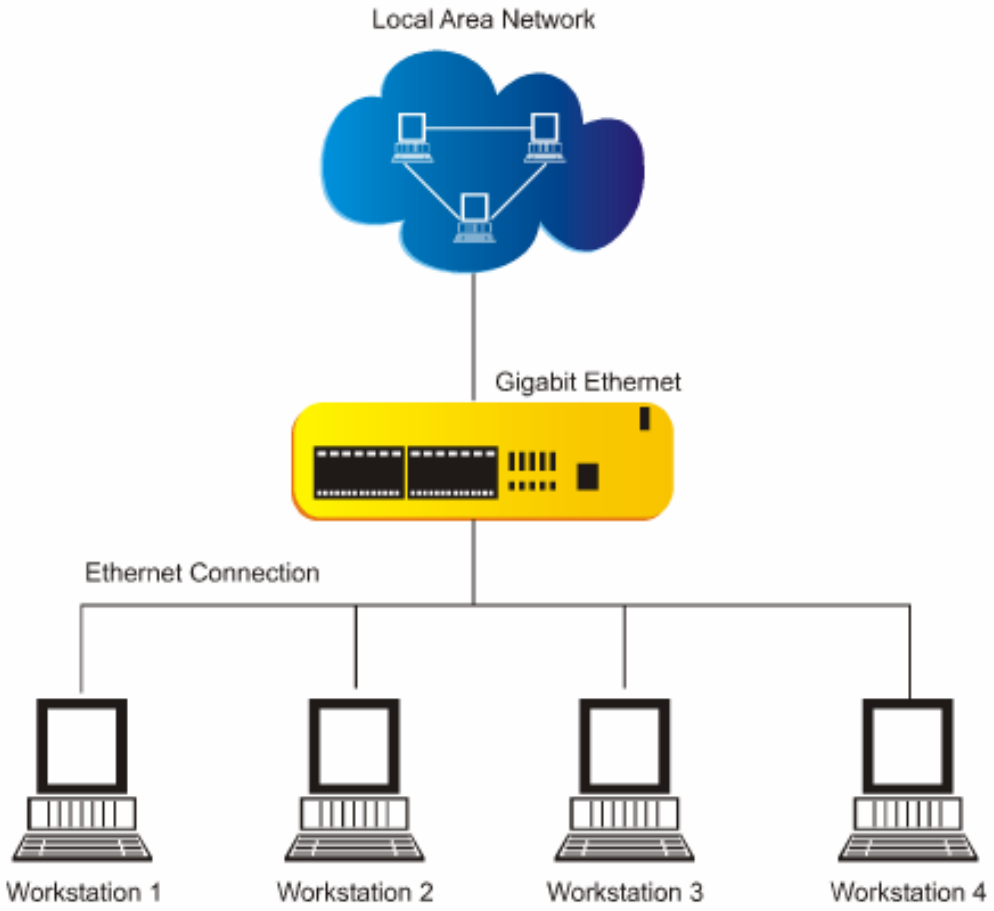
https://www.sciencedirect.com/topics/computer-science/workstation-network

Networks of Workstations

- High-speed networks and rapidly improving microprocessor performance make ***networks of workstations*** an increasingly ***appealing system for parallel computing***.
- By relying solely on **commodity hardware** and **software**, networks of workstations offer parallel processing at a relatively low cost.
- ***Commodity hardware*** (unlike ***purpose-built hardware***), sometimes known as ***off-the-shelf hardware***, is a computer device or IT component that is relatively ***inexpensive***, ***widely available*** and basically ***interchangeable with other hardware*** of its type.

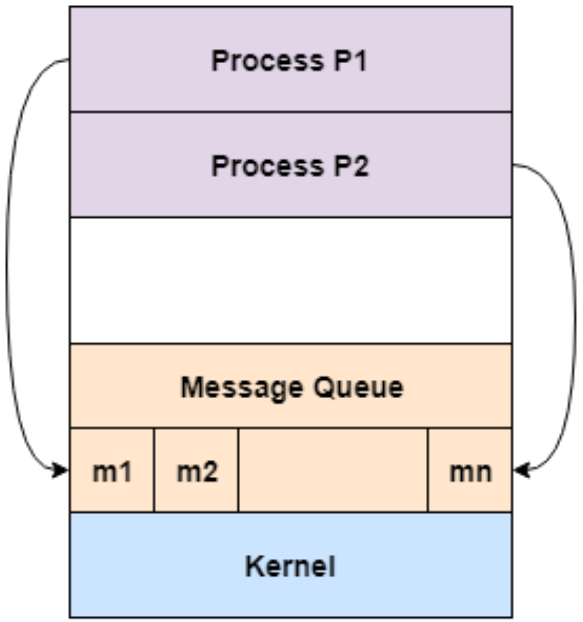
Architecture Overview

- **Workstations (Nodes)**
  - Independent computers with full OS
  - Perform local and distributed computation
- **Network Interconnect**
  - High-speed LAN (Ethernet, Myrinet, InfiniBand)
  - Provides data transfer and communication
- **Middleware/Software Layer**
  - Handles communication, synchronization, and resource management
  - Examples: MPI, PVM, RPC frameworks
- **Resource Manager / Scheduler**
  - Allocates jobs and balances load dynamically



Communication Mechanisms

- **Message Passing Paradigm:**
  - Processes communicate via **explicit messages**.
  - Communication handled using **network protocols (TCP/IP)**.
  - Libraries and tools:
    - **MPI (Message Passing Interface)**
    - **PVM (Parallel Virtual Machine)**
    - **RPC (Remote Procedure Calls)**
- **Performance Factors:**
  - **Latency:** Time to start message transmission.
  - **Bandwidth:** Amount of data transmitted per second.
  - **Overheads:** Protocol stack, buffering, and synchronization delays.



Message Passing Model

Implementation Differences between MPI, PVM, and RPC (1)

Aspect	MPI (Message Passing Interface)	PVM (Parallel Virtual Machine)	RPC (Remote Procedure Call)
Basic Nature	A **standardized library specification** (not a single implementation). Implemented as optimized libraries (e.g., MPICH, OpenMPI).	A **software framework/daemon-based system** that creates a "virtual machine" across heterogeneous nodes.	A **communication protocol abstraction** that enables procedure calls across networked systems.
Process Management	MPI **does not manage processes** — the processes are typically started together (e.g., via mpirun or mpiexec). It assumes all processes are already running and are part of the MPI environment.	PVM **manages processes dynamically** through its **pvmd daemon**. It can spawn, add, or remove hosts and processes at runtime.	RPC has a **client–server model**. The server provides callable procedures, and clients invoke them on demand. Process management is external (handled by OS or runtime).
Communication Mechanism	Direct **point-to-point** or **collective communication** between processes using low-level protocols (TCP/IP, Infiniband, shared memory).	Communication handled by **PVM daemon** which intermediates message routing between tasks.	Underlying communication uses **request–reply** model. Message passing is hidden — stub code (marshalling/unmarshalling) handles serialization.
Architecture Type	**Peer-to-peer** — all processes are equal participants in computation.	**Master-slave** or **virtual machine** model with centralized control through pvmd daemon.	**Client–server** — one side provides services, the other requests them.
Performance	Very **high performance** — uses optimized communication libraries (zero-copy, RDMA).	**Lower performance** due to daemon overhead (indirect communication path).	**High latency** compared to MPI/PVM, as it's designed for infrequent service calls, not high-frequency parallel communication.

Implementation Differences between MPI, PVM, and RPC (2)

Scalability	Scales efficiently to **thousands of processes** (supercomputers, clusters).	Scales well to moderate-size heterogeneous systems (hundreds of nodes).	Limited scalability — meant for **distributed systems**, not parallel HPC workloads.
Programming Model	Explicit **message passing** via API calls like MPI_Send, MPI_Recv, MPI_Bcast, etc.	Similar API (e.g., pvm_send, pvm_recv), but with extra features for dynamic host management.	Implicit communication — looks like a normal function call (remote_add(x,y)), implemented via **stubs and skeletons**.
Fault Tolerance	Limited built-in fault recovery; failures usually abort the job (though some implementations support fault tolerance).	Supports **dynamic fault recovery** — failed hosts can be replaced or restarted.	Built-in fault reporting (return codes, exceptions) but **no automatic recovery**.
Implementation Complexity	Implemented as an **optimized library layer** tightly integrated with the OS/network stack.	Implemented as a **middleware layer** with background daemons managing tasks and messages.	Implemented as **middleware + compiler-generated stubs**, handling data serialization/deserialization.
Use Case in PDC	High-performance parallel computing (HPC), clusters, supercomputers.	Networks of heterogeneous workstations (NOW), grid-style distributed computing.	Distributed applications, microservices, and networked systems (not HPC).

Berkeley NOW Project (Case Study)

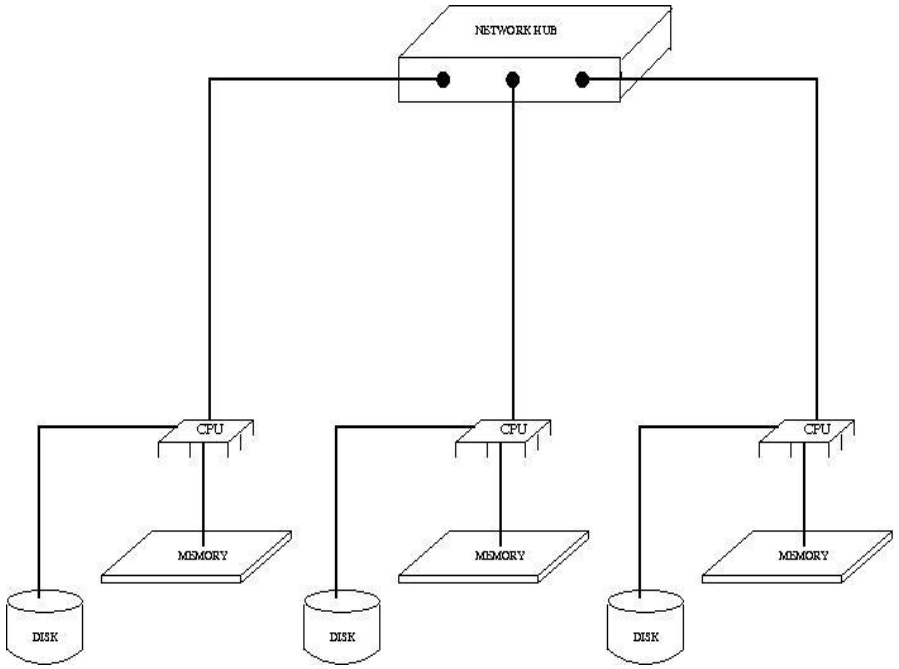
- **University of California, Berkeley (1990s)**
- **Goal:** To build a high-performance computing platform using **commodity workstations and a fast network**.
- **Key Innovations:**
  - High-speed **Myrinet** network interconnect
  - **NOW-FS (Global File System)** for shared access
  - **Efficient process migration** and load balancing
  - Developed **Active Messages** — a low-latency communication mechanism
- **Impact:**
  - Demonstrated feasibility of using workstation networks for parallel workloads
  - Inspired Beowulf clusters and early grid computing

NOW vs Multiprocessors

Feature	Multiprocessor	Network of Workstations
Architecture	Shared memory	Distributed memory
Communication	Through shared bus	Through network links
Scalability	Limited (due to bus contention)	High (add more nodes)
Fault Tolerance	Single point of failure	Independent nodes
Cost	High	Low
Example	Intel Xeon SMP	Berkeley NOW, LAN clusters

Concept of Distributed Memory

- In a **distributed memory system**, each processor (workstation) has its **own private memory**.
- No global address space — memory cannot be directly accessed by other nodes.
- All data sharing occurs through **explicit message passing**.
- Each node executes part of a program and communicates results when necessary.
- **Example:** A NOW of 10 Linux PCs connected by Gigabit Ethernet forms a distributed memory system.



Data Distribution and Partitioning

- **Challenges:**
  - How to divide large data across multiple nodes efficiently.
  - How to minimize communication overhead.
- **Strategies:**
  - **Block Distribution** – Divide array into equal-sized blocks assigned to nodes.
  - **Cyclic Distribution** – Elements distributed round-robin style to balance load.
  - **Block-Cyclic Distribution** – Combines benefits of both for irregular workloads.

Block Distribution

- The dataset (e.g., an array or matrix) is **divided into contiguous blocks**, and each block is assigned to a specific node (processor or workstation).
- Each processor owns a **complete block** of data (responsible for computing results on its portion independently).
- Suppose we have an array of 16 elements: A = [1, 2, 3, ..., 16], With 4 processors (P0–P3) and **block size = 4**, distribution becomes:
- P0 → [1,2,3,4], P1 → [5,6,7,8], P2 → [9,10,11,12], P3 → [13,14,15,16]
- **Limitations**
  - Load imbalance may occur when different parts of data take unequal processing time.
  - Not ideal for irregular workloads or when all processors need parts of all data.

Cyclic Distribution for Load Balancing

- Elements are distributed among processors in a **round-robin (cyclic) fashion**.
- Each node receives **every nth element**, ensuring that work is evenly spread even if tasks take varying amounts of time.
- Using the same array A = [1, 2, 3, ..., 16] with 4 processors:
- P0 → [1,5,9,13], P1 → [2,6,10,14], P2 → [3,7,11,15], P3 → [4,8,12,16]
  - Element i is assigned to processor i mod p.
- **Limitations:**
  - **Increased communication** — since processors need non-local data more often.
  - **Cache locality** decreases — as elements handled by each node are dispersed in memory.

Block-Cyclic Distribution – Balancing Load and Locality

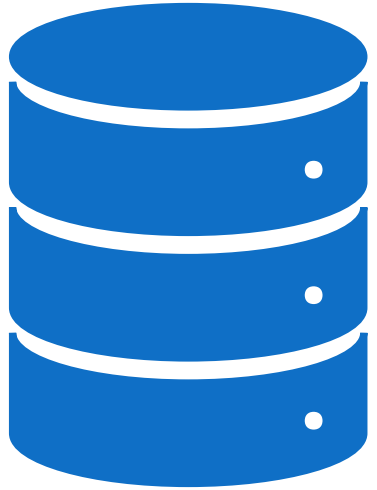
- A hybrid of **Block** and **Cyclic** distribution strategies.
- Data is divided into **small blocks**, and these blocks are then distributed cyclically among processors.
- Balances **load distribution** (like cyclic) while maintaining **data locality** (like block).
- Consider 16 elements, 4 processors, and block size = 2:
- P0 → [1,2], [9,10], P1 → [3,4], [11,12], P2 → [5,6], [13,14], P3 → [7,8], [15,16]
  - Blocks of 2 elements are cyclically assigned to processors.
- Advantages:
  - Load-balanced like cyclic distribution.
  - Preserves spatial locality by grouping elements in small blocks.
  - Highly flexible for both regular and irregular computations.

Process Communication

- Each node executes a process that communicates via **message passing**.
- Communication mechanisms:
  - **Point-to-point:** send/receive between two processes.
  - **Collective:** broadcast, gather, scatter, reduce operations.
- Implemented using libraries such as:
  - **MPI_Send(), MPI_Recv()** (MPI)
  - **pvm_send(), pvm_recv()** (PVM)

Performance Issues in Distributed Memory

- **Network latency:** time to initiate communication.
- **Bandwidth:** rate of data transfer.
- **Granularity:** size of tasks—fine-grained tasks → more communication.
- **Synchronization overhead:** waiting for other nodes.



Programming Issues in Distributed Memory

- The key issue in ***programming*** distributed memory systems is ***how to distribute the data over the memories***.
- Depending on the problem solved, the data can be ***distributed statically***, or it can be ***moved through the nodes***.
- Data can be ***moved on demand***, or data can be ***pushed to the new nodes in advance***.

Programming Issues in Distributed Memory

- Data can be ***kept statically*** in nodes if ***most computations happen locally***, and only changes on edges have to be reported to other nodes.
- An example of this is simulation where ***data is modeled*** using a ***grid*** (non-interactive workloads), and ***each node simulates a small part of the larger grid***.
- On every iteration, nodes **inform** all neighboring nodes of the new edge data.

Shared vs. Distributed Memory

- There are two kinds of multiple-processor systems exist:
  - **Multi-Processors** (Shared/ Distributed)
  - **Multi-Computers** (Distributed)
- In a **multi-processor** two or more CPUs **share a common main memory**.
- Any ***process*** on any ***processor***, can read or write any word in shared memory, simply by moving data to or from the desired location.
- In a **multi-computer**, in contrast, **each CPU has its own private memory**. Nothing is shared.

Shared vs. Distributed Memory

- To make an **agriculture analogy**, a multiprocessor is a system with a **herd of sheep** (***processes***) eating from a **single feeding through** (***shared memory***).
- A **multicomputer**, on the other hand, is a design in which each sheep has its own **private feeding through (*distributed memory*)**.

Summary & Discussion

- Distributed memory = each node has **private memory** and communicates via **messages**.
- NOW systems are **distributed-memory architectures** implemented using networked workstations.
- Achieve parallelism through **data partitioning, message passing**, and **synchronization**.
- Trade-off between **performance** and **programming complexity**.

Thanks


Sometimes you will never know the value of a moment until it becomes a memory
-Dr. Suess
facebook.com/xaviersashes